

No Human In Loop (NHIL) Programming

LLM Indiv Project 2
Brian Cheng

Abstract

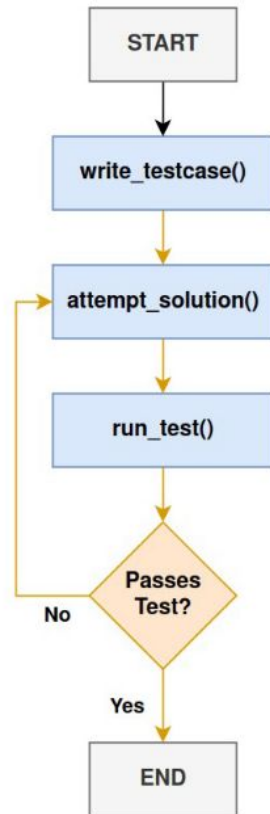
- Automated self-verifying programming framework built on LangGraph, LangChain, and ChatOpenAI.
 - Driving an LLM through a cycle of unit-test generation, solution synthesis, and test execution, the system
 - Progressively refines its code until it passes all tests or reaches a retry limit.
- We illustrate this process on a simple **Arithmetic-expression evaluator** problem, showing how syntax and logic errors are detected and repaired over three attempts.

Motivation

- Reliability through Testing.
- LLM generated code is often wrong.
 - Can fail compilation
 - Can pass compilation but fail to implement desired behavior
 - Can an LLM detect flaws in its own responses without human intervention?
 - Adversarial LLM agents?
- Iterative Self-Repair
 - Embed unittest Testcase into the design cycle
 - Automatic feedback from testcase failure drives targeted fixes
 - Reduces manual intervention
 - Accelerates convergence to a working code solution

Framework

- Core Components
 - ChatOpenAI(model="gpt-4o")
 - Pydantic to enforce structured input/outputs between nodes
- StateGraph Nodes
 - write_testcase()
 - attempt_solution()
 - run_test()
- Control Logic
 - **Pass all tests** => terminate loop
 - **Failure** => loop back to attempt_solution() with error
 - **Maxed_failures** => terminate with failure



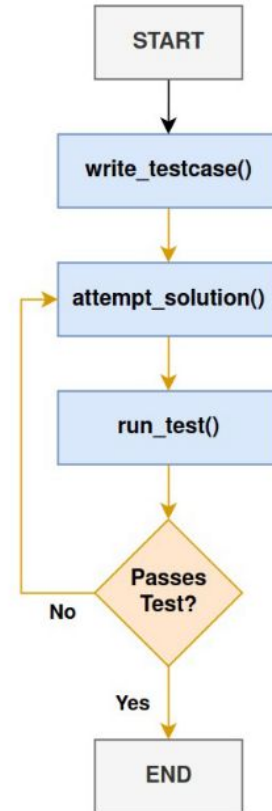
StateGraph Builder

```
builder = StateGraph(GraphState)

builder.add_node("write_testcase", write_testcase)
builder.add_node("attempt_solution", attempt_solution)
builder.add_node("run_test", run_test)

builder.set_entry_point("write_testcase")
builder.add_edge("write_testcase", "attempt_solution")
builder.add_edge("attempt_solution", "run_test")
builder.add_conditional_edges(
    "run_test",
    decide_next_node,
    {
        "end" : END,
        "attempt_solution" : "attempt_solution"
    }
)

graph = builder.compile()
```



Demonstration: Arithmetic expression evaluator

```
Problem:
Implement a Python function
    evaluate(expression: str) → int
that takes an arithmetic expression string and returns its integer result.-

Your evaluator must support:
    Non-negative integers
    Binary operators +, -, *, / (with integer division truncating toward zero)
    Parentheses ( and ) at arbitrary nesting depth
    Arbitrary whitespace mixed in

Constraints:
    Do not use Python's built-in eval() (or ast.literal_eval, etc.).
    Must correctly handle operator precedence and left-associativity.
    Division by zero may raise an exception.

Unit tests:
Write a unittest.TestCase suite in test/test_calculator.py that covers:
    Simple expressions like "3+2*2" → 7
    Whitespace variants: " 3 /2 " → 1
    Nested parentheses: "(2+3)*(4-1)" → 15
    Division with negatives: "-7/3" → -2 (trunc toward zero)
    Deeply nested: "((1+2)*((3-4)+5))/2" → 6
    Invalid syntax cases (you can assert that they raise an exception)
```

```
f1 = open("user_problem.txt", "r")
user_problem = f1.read()
f1.close()
initial_prompt = user_problem
print(initial_prompt)

initial_state = {
    "attempts": 0,
    "messages": [HumanMessage(content=initial_prompt)],
    "testcase": "",
    "solution": "",
    "error": "yes"
}

# Traverse the graph
for ev in graph.stream(initial_state, stream_mode="events"):
    print(ev)
```

user_problem.txt

Write Testcase

```
test_prompt = messages + [
    (
        "system",
        f"""
        Design cycle: {attempts}
        First, write a unittest TestCase that
        will be used to verify your solution
        to the user defined problem.-
        The testcase must call the solution
        function.-
        The testcase must contain the main
        function.-
        Your entire response will be
        interpreted as Python code.-
        Adhere to Python syntax and use
        comment blocks for comments.-
        Do not work on the solution yet.-
        Do not import any external libraries.-

        At the top of the file, add:
        | from solution import evaluate

        And make sure every 'evaluate(...)'
        call is syntactically complete (no
        trailing commas).

        """,
    )
]
```

```
# test_calculator.py
import unittest
from solution import evaluate    • Import "solution" could not be resolved

class TestCalculator(unittest.TestCase):
    ...

    def test_simple_expression(self):
        # Testing simple expression without whitespace
        self.assertEqual(evaluate("3+2*2"), 7)

    def test_whitespace_variants(self):
        # Testing expression with whitespaces
        self.assertEqual(evaluate(" 3 /2 "), 1)

    def test_nested_parentheses(self):
        # Testing expression with nested parentheses
        self.assertEqual(evaluate("(2+3)*(4-1)"), 15)

    def test_division_with_negatives(self):
        # Testing division with negative numbers, expect truncation towards zero
        self.assertEqual(evaluate("-7/3"), -2)

    def test_deeply_nested_parentheses(self):
        # Testing deeply nested expression
        self.assertEqual(evaluate("((1+2)*((3-4)+5))/2"), 6)

    def test_invalid_syntax(self):
        # Testing invalid syntax, expecting exception
        with self.assertRaises(Exception):
            evaluate("""
            3 + + 2
            """)

if __name__ == '__main__':
    unittest.main()
```

attempt_01

```
ops.append(token)
elif token == ')':
    while ops and ops[-1] != '(': # Resolve until matching '('
        apply_last_operator()
    ops.pop() # Remove '(' from ops
elif token in "+-*/":
    while (ops and ops[-1] != "("): # "(" was not closed
        dex < len(self.tokens):
            token = self.tokens[dex]
            dex += 1
if isinstance(token, int):
    values.append(token)
elif token == '(':
    ops.append(token)
```

```
E
=====
ERROR: test (unittest.loader._FailedTest.test)
-----
ImportError: Failed to import test module: test
Traceback (most recent call last):
  File "/home/bcheng/workspace/tools/anaconda3/envs/langchain/lib/python3.12/unittest/loader.py", line 394, in
    _find_test_path
    module = self._get_module_from_name(name)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/bcheng/workspace/tools/anaconda3/envs/langchain/lib/python3.12/unittest/loader.py", line 337, in
    _get_module_from_name
    __import__(name)
  File "/home/bcheng/workspace/dev/langchain/proj2/test/test.py", line 3, in <module>
    from solution import evaluate
  File "/home/bcheng/workspace/dev/langchain/proj2/src/attempt_01/solution.py", line 91
    while (ops and
          ^
SyntaxError: '(' was never closed

-----
Ran 1 test in 0.000s

FAILED (errors=1)
```


attempt_02

```
8 def evaluate(self):
7     """Evaluate the expression given in tokens."""
6     values = [] # Stack for numbers
5     ops = [] # Stack for operators
4
3     def apply_last_operator():
2         """Apply the last operator to the top two values on the stack."""
1         right = values.pop()
73        left = values.pop()
4        op = ops.pop()
2        values.append(self.apply_operator(left, right, op))
3
4        index = 0
5        while index < len(self.tokens):
```

.E....

=====

ERROR: test_division_with_negatives (test.TestCalculator.test_division_with_negatives)

Traceback (most recent call last):

File "/home/bcheng/workspace/dev/langchain/proj2/test/test.py", line 21, in test_division_with_negatives

self.assertEqual(evaluate("-7/3"), -2)

^^^^^^^^^^^^^^^^^^^^

File "/home/bcheng/workspace/dev/langchain/proj2/src/attempt_02/solution.py", line 109, in evaluate

return evaluator.compute()

^^^^^^^^^^^^^^^^^^^^

File "/home/bcheng/workspace/dev/langchain/proj2/src/attempt_02/solution.py", line 104, in compute

return self.evaluate()

^^^^^^^^^^^^^^^^

File "/home/bcheng/workspace/dev/langchain/proj2/src/attempt_02/solution.py", line 98, in evaluate

apply_last_operator()

File "/home/bcheng/workspace/dev/langchain/proj2/src/attempt_02/solution.py", line 73, in apply_last_operator

left = values.pop()

^^^^^^^^^^^^

IndexError: pop from empty list

Ran 6 tests in 0.001s

FAILED (errors=1)

attempt_03

.....

Ran 6 tests in 0.000s

OK

Future Work (1/3)

- **Improved Prompt Construction**

- Design cycle iteration
- Current chat history, including prior testcases, solutions, and error messages
 - Each solution and error message is **appended** to the chat history
 - **Linear memory growth, scales poorly**, considering the LLM's context windows.
 - Future work: **summarize** each solution and error message and save to file.
 - Allow LLM to “**bookmark**” past design cycles.
 - Allow LLM to **choose a bookmark and read** a past design cycle's **files** to fill the LLM's short term context window with only the **most relevant information**.
- Precise system instructions
 - “Respond only with valid compilable Python code”
 - “Use comment blocks for any code explanations”
 - “Import xyz libraries before writing the code”

Future Work (2/3)

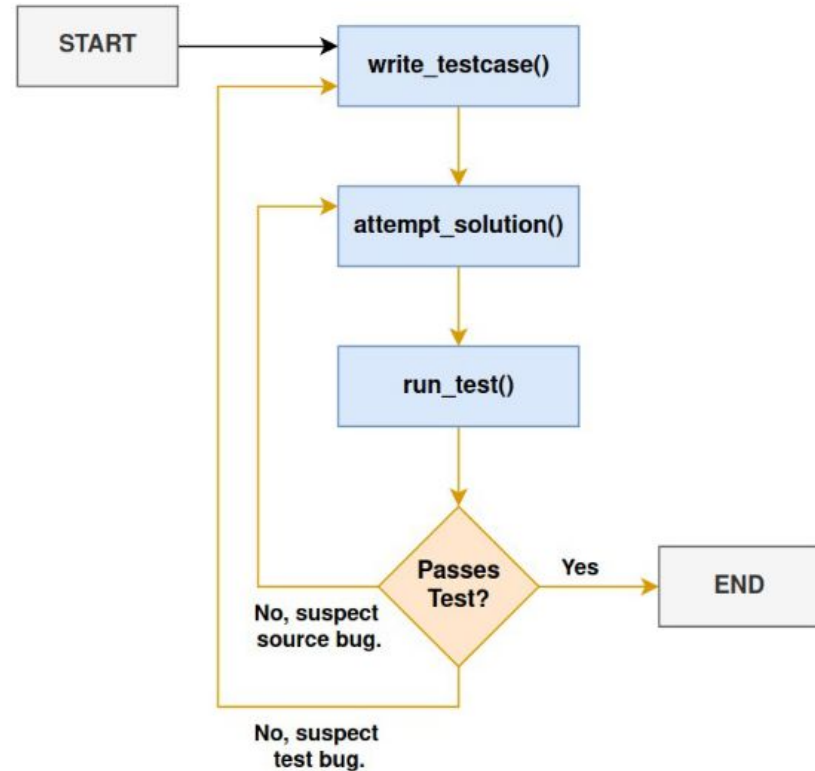
- **LangGraph Shell Agent**

- Allow LLM to access the OS directory
- Allow LLM to **execute shell commands directly**.
 - Allow LLM to manage a small **sandbox directory environment**.
 - Allow LLM to **execute** python and conda **commands directly**.
 - Allow LLM to write and execute shell **batch scripts**.
 - Allow LLM to **install external Conda / Python libraries** and other tools.
 - Allow LLM to **read and write to files directly**.
- Powerful, but **potentially dangerous**
 - LLM could act erroneously or maliciously and damage the user's environment or the system.
 - Need to construct a secure isolated virtual environment for the LLM agent.

Future Work (3/3)

- **Testcase Correction Flow**

- It is very possible the LLM makes a **mistake in writing the Testcase**.
- Find a way to allow LLM to decide if the program failed due to **solution bug** OR **testcase bug**
- Add an additional conditional edge on the `run_test()` node going into the `write_testcase()` node.
- Allow iterative self-correction for both solution and testcase



Proposed StateGraph